



unc



FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

TP1 — Implementación de una ALU de 8 bits sobre Basys3

Materia:	Arquitectura de Computadoras
Carrera:	Ingeniería en Computación
Alumnos:	García, Lautaro Misael Renaudo Gaggioli, Valentino
Fecha:	Septiembre, 2026

Índice

1. Objetivos
2. Descripción general del sistema
3. Arquitectura de la implementación
4. Operaciones implementadas
5. Flags
6. Integración mediante top
7. Interfaz física con la Basys3
8. Mapeo de pines
9. Verificación mediante simulación
10. Decisiones de diseño
11. Conclusiones

1. Objetivos

El objetivo del trabajo es diseñar e implementar una Unidad Aritmético-Lógica (ALU) sobre una FPGA Digilent Basys3. La ALU debe operar sobre dos operandos de 8 bits y permitir seleccionar entre ocho operaciones aritméticas, lógicas y de desplazamiento mediante un código de operación de 6 bits.

Además de generar el resultado, el diseño incorpora dos flags:

- **Zero:** indica que el resultado de la operación es igual a cero.
- **Overflow:** indica un desbordamiento en complemento a dos para las operaciones aritméticas ADD y SUB.

La interfaz física utiliza los switches de la placa para ingresar los datos y los pulsadores para capturar los operandos y la operación. El resultado y los flags se visualizan mediante los LEDs.

El proyecto también incluye testbenches para verificar individualmente la ALU, el registro de captura por botón y la integración completa del sistema.

2. Descripción general del sistema

El sistema está compuesto por tres bloques principales, los cuales permiten ingresar dos operandos y un código de operación utilizando los switches de la Basys3.

1. **Registro del operando A.**
2. **Registro del operando B.**
3. **Registro del código de operación.**
4. **ALU combinacional.**

El módulo `top` realiza la interconexión entre estos bloques y asigna las señales internas a los recursos físicos de la Basys3.

Los switches constituyen un bus de entrada compartido. Cuando se presiona un pulsador, el valor presente en los switches se almacena en el registro correspondiente.

La secuencia normal de utilización es:

1. Colocar el valor del operando A en `sw[7:0]` y presionar `btnU`.
2. Colocar el valor del operando B en `sw[7:0]` y presionar `btnD`.
3. Colocar el código de operación en `sw[5:0]` y presionar `btnC`.
4. Observar el resultado en `LED[7:0]`.
5. Observar las flags `Zero` y `Overflow` en `LED[14]` y `LED[15]`.

El botón `btnR` realiza un reset síncrono de los registros.

3. Arquitectura de la implementación

La implementación se divide en los módulos `alu`, `btn_reg` y `top`.

3.1. Módulo `btn_reg`

El módulo `btn_reg` implementa un registro parametrizable utilizado para almacenar los operandos y el código de operación.

Sus entradas principales son:

- `i_clk`: reloj de 100 MHz de la Basys3.
- `i_rst`: reset síncrono.
- `i_btn`: pulsador utilizado como señal de captura.
- `i_sw`: valor presente en los switches.

Su salida es:

- `o_data`: valor almacenado.

La captura se realiza detectando el flanco ascendente de `i_btn`. Para ello, el módulo almacena en `r_btn_prev` el estado anterior del pulsador.

La condición de captura es:

```
i_btn = 1
i_btn anterior = 0
```

De esta forma, un pulsador mantenido presionado no provoca múltiples capturas. El valor queda almacenado hasta que se produzca una nueva captura o un reset.

El parámetro `WIDTH` permite reutilizar el módulo tanto para los operandos de 8 bits como para el código de operación de 6 bits.

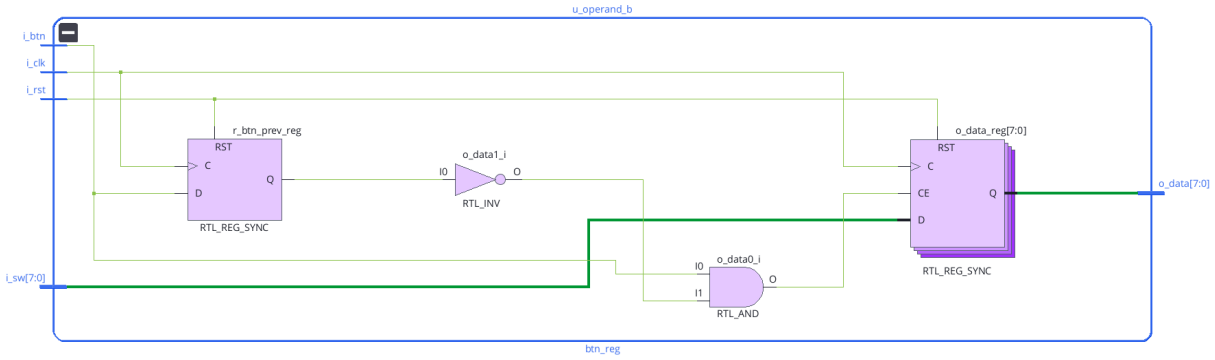


Figure 1: Esquemático RTL de btn_reg

3.2. Módulo alu

La ALU recibe dos operandos de ancho parametrizable y un código de operación de 6 bits.

Sus salidas son:

- **o_result**: resultado de la operación.
- **o_overflow**: flag de overflow.
- **o_zero**: flag de resultado cero.

La lógica de selección de operaciones se implementa mediante una estructura **case**, por lo que la ALU funciona como lógica combinacional.

El ancho de datos se define mediante el parámetro NB_DATA, cuyo valor utilizado en el top es 8.

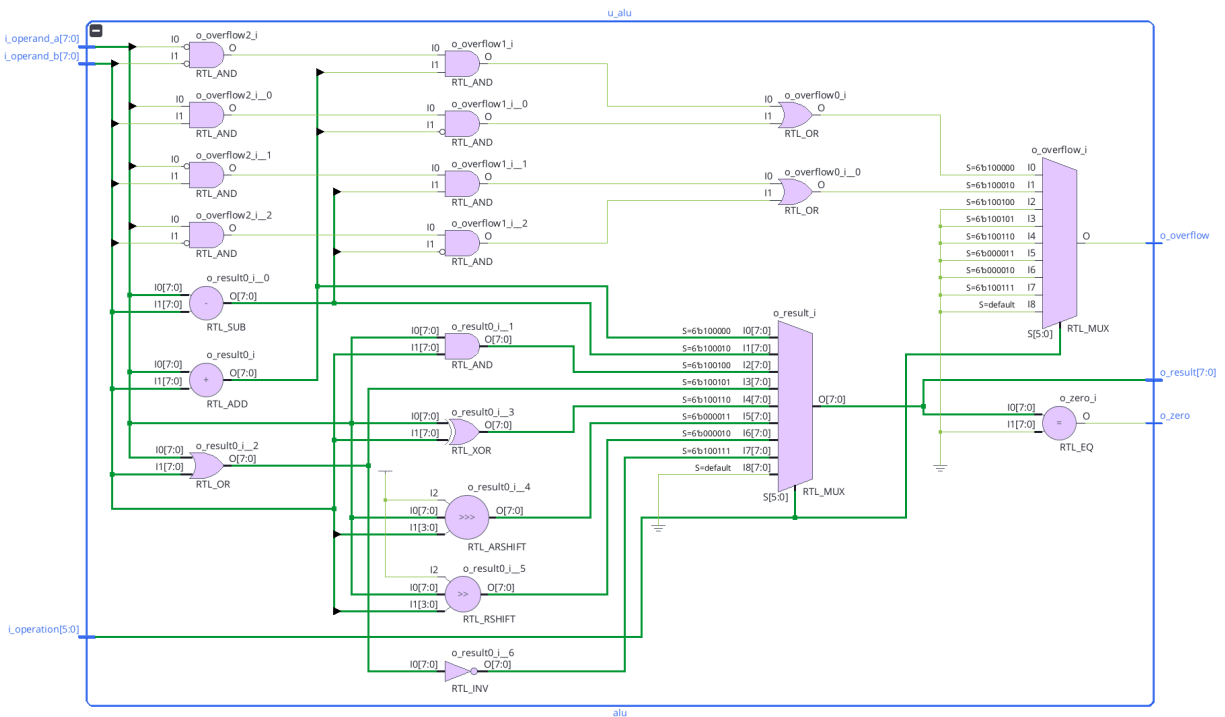


Figure 2: Esquemático RTL de alu

4. Operaciones implementadas

La ALU implementa las ocho operaciones requeridas mediante los siguientes códigos:

Operación	Código	Descripción
ADD	100000	Suma de A y B
SUB	100010	Resta de A y B
AND	100100	AND bit a bit
OR	100101	OR bit a bit
XOR	100110	XOR bit a bit
SRA	000011	Desplazamiento aritmético a derecha
SRL	000010	Desplazamiento lógico a derecha
NOR	100111	NOR bit a bit

4.1. Operaciones aritméticas

ADD realiza:

$A + B$

mientras que SUB realiza:

$A - B$

El resultado se mantiene en el ancho configurado de la ALU. Para una implementación de 8 bits, esto implica que los bits que exceden dicho ancho no forman parte de `o_result`.

4.2. Operaciones lógicas

Las operaciones AND, OR, XOR y NOR se realizan bit a bit sobre los dos operandos.

Por ejemplo:

A = 10101010

B = 01010101

A AND B = 00000000

A OR B = 11111111

A XOR B = 11111111

A NOR B = 00000000

4.3. Desplazamiento aritmético

SRA realiza un desplazamiento hacia la derecha conservando el signo del operando A. Para ello, A se interpreta explícitamente como un valor con signo mediante `$signed`.

La cantidad de desplazamiento se obtiene a partir de los bits bajos de B.

4.4. Desplazamiento lógico

SRL realiza un desplazamiento lógico hacia la derecha de A. En este caso se introducen ceros por la izquierda.

La cantidad de desplazamiento también se obtiene a partir de los bits bajos de B.

4.5. Operación por defecto

Si el código de operación no coincide con ninguna de las ocho operaciones definidas, la ALU genera:

```
resultado = 0
overflow = 0
```

Como consecuencia, la flag **Zero** también queda activa.

5. Flags

5.1. Flag Zero

La flag **Zero** se genera comparando el resultado de la ALU con cero:

```
o_zero = (o_result == 0)
```

Por lo tanto:

- **Zero** = 1 cuando **o_result** es cero.
- **Zero** = 0 cuando **o_result** es distinto de cero.

La flag se aplica a todas las operaciones, no solamente a **ADD** y **SUB**.

Por ejemplo:

```
A = 0xAA
B = 0xAA
A XOR B = 0x00
Zero = 1
```

5.2. Flag Overflow

La flag **Overflow** se utiliza exclusivamente en las operaciones **ADD** y **SUB**.

Para una ALU de 8 bits con representación en complemento a dos, el rango de valores con signo es:

$$-128 \leq x \leq 127$$

En **ADD**, se detecta overflow cuando:

- dos operandos positivos producen un resultado negativo; o
- dos operandos negativos producen un resultado positivo.

La condición implementada es equivalente a:

```
(~signA & ~signB & signResult) |
( signA &  signB & ~signResult)
```

En **SUB**, se detecta overflow cuando:

- un operando positivo menos uno negativo produce un resultado negativo; o
- un operando negativo menos uno positivo produce un resultado positivo.

La condición implementada es equivalente a:

```
(~signA & signB & signResult) |
( signA & ~signB & ~signResult)
```

Por ejemplo:

```
0x7F + 0x01 = 0x80
```

El valor matemático es 128, que no puede representarse en 8 bits con signo. Por lo tanto:

```
resultado = 0x80
Overflow = 1
```

El diseño conserva el resultado de 8 bits producido por la operación; no implementa saturación.

5.3. Carry

No se implementa una flag **Carry** en este diseño. La condición de desbordamiento considerada para las operaciones aritméticas es específicamente **Overflow** en representación signed.

Esto diferencia el desbordamiento de complemento a dos (**Overflow**) del acarreo correspondiente a una interpretación unsigned (**Carry**).

6. Integración mediante top

El módulo **top** conecta los registros de entrada con la ALU.

Se utilizan tres instancias de **btn_reg**:

- Una instancia de ancho 8 para el operando A.
- Una instancia de ancho 8 para el operando B.
- Una instancia de ancho 6 para el código de operación.

Las conexiones son:

```
sw[7:0] → registro A
sw[7:0] → registro B
sw[5:0] → registro OP
```

La ALU recibe los tres valores almacenados y genera continuamente el resultado y las flags.

Por tratarse de una ALU combinacional, el resultado se actualiza automáticamente cuando cambia cualquiera de sus entradas registradas.

7. Interfaz física con la Basys3

7.1. Switches

Aunque la entrada **sw** posee 16 bits, el diseño utiliza:

- **sw[7:0]** para los operandos.
- **sw[5:0]** para el código de operación.

Los switches **sw[15:8]** no intervienen en el funcionamiento del sistema.

7.2. Pulsadores

Pulsador	Función
btnU	Capturar operando A
btnD	Capturar operando B
btnC	Capturar código de operación
btnR	Reset síncrono

La captura se realiza sincronizada con el reloj de 100 MHz mediante detección del flanco ascendente del estado del botón.

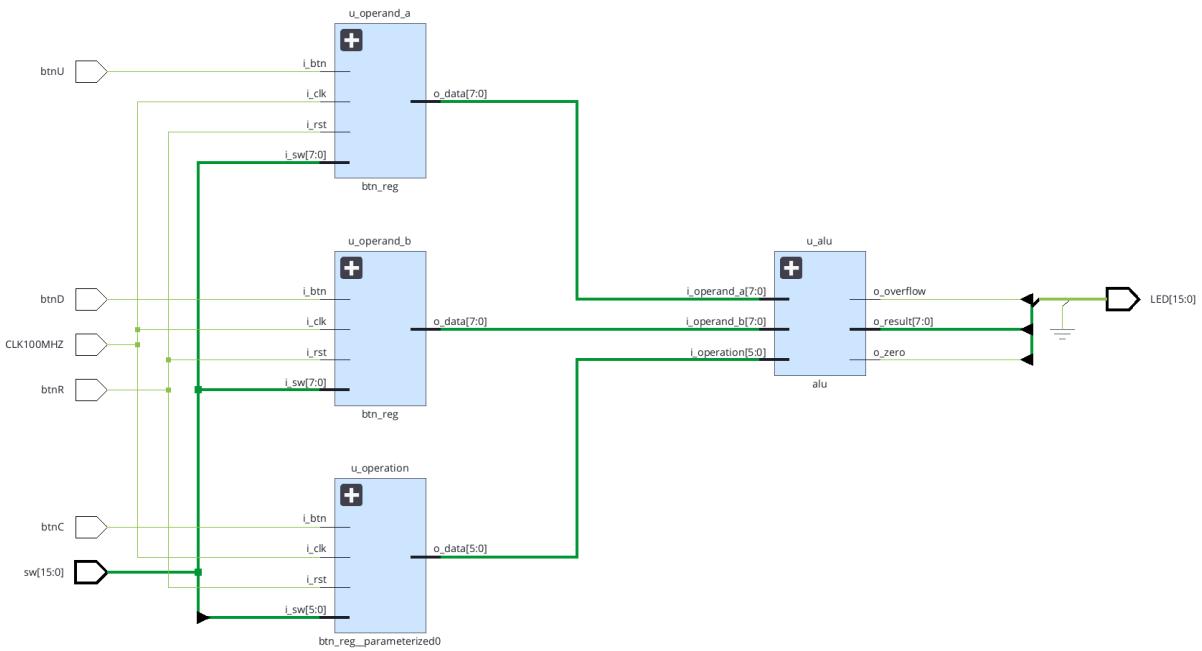


Figure 3: Esquemático RTL de `top`

7.3. LEDs

La salida de 16 bits se distribuye de la siguiente forma:

LEDs	Función
LED[7:0]	Resultado de la ALU
LED[13:8]	Sin utilizar, forzados a cero
LED[14]	Zero
LED[15]	Overflow

Por ejemplo, si el resultado es `0x08`, los ocho LEDs inferiores representan:

`00001000`

Si se produce overflow con `0x7F + 0x01`, se obtiene:

`LED[15] = 1`

`LED[14] = 0`

`LED[7:0] = 10000000`

8. Mapeo de pines

El archivo `Basys3_Master.xdc` establece la correspondencia entre las señales del módulo `top` y los pines físicos de la FPGA.

El reloj de 100 MHz está conectado al pin `W5`.

Los switches y LEDs utilizan las asignaciones estándar de la Basys3 incluidas en el archivo de constraints.

Los pulsadores utilizados por el diseño son:

Señal	Pulsador	Pin
btnC	Center	U18
btnU	Up	T18
btnD	Down	U17
btnR	Right	T17

La señal **btnL** permanece definida en el archivo maestro de constraints, pero no forma parte de las entradas del módulo **top** y no se utiliza en el diseño.

El archivo también contiene las asignaciones de los 16 switches y los 16 LEDs correspondientes a la Basys3.

9. Verificación mediante simulación

El proyecto contiene tres testbenches:

- **tb_alu.sv**
- **tb_btn_reg.sv**
- **tb_top.sv**

Las pruebas se ejecutan utilizando Icarus Verilog mediante el script **test.sh**.

9.1. Testbench de la ALU

tb_alu.sv verifica las ocho operaciones y las flags **Zero** y **Overflow**.

Se incluyen casos normales y casos límite.

Los casos de prueba abarcan:

- ADD normal.
- ADD con overflow positivo.
- ADD de dos valores negativos con overflow.
- ADD sin overflow con resultado cero.
- SUB normal.
- SUB con overflow.
- SUB con resultado cero.
- AND normal y con resultado cero.
- OR normal y con resultado cero.
- XOR normal y con resultado cero.
- NOR normal y con resultado cero.
- SRL con distintos desplazamientos.
- SRA con operandos positivos y negativos.
- Código de operación no definido.

El archivo contiene **23 casos de prueba efectivos**.

9.2. Testbench de btn_reg

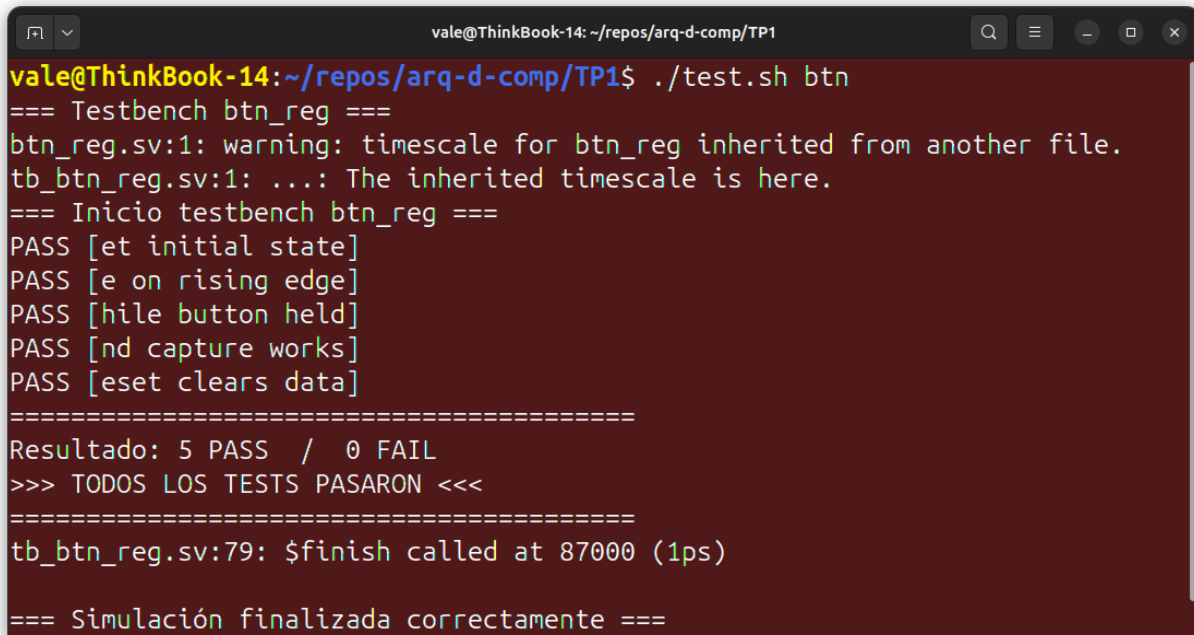
tb_btn_reg.sv verifica:

1. Estado inicial después del reset.
2. Captura de un valor cuando se detecta el flanco ascendente del botón.
3. Ausencia de recaptura mientras el botón permanece presionado.
4. Funcionamiento de una segunda captura después de liberar y volver a presionar el botón.

```
vale@ThinkBook-14: ~/repos/arq-d-comp/TP1
vale@ThinkBook-14:~/repos/arq-d-comp/TP1$ ./test.sh alu
=== Testbench ALU ===
alu.sv:1: warning: timescale for alu inherited from another file.
tb_alu.sv:1: ...: The inherited timescale is here.
=== Inicio testbench ALU (NB_DATA=8) ===
PASS [ADD normal]
PASS [ ovf pos+pos=neg]
PASS [neg+neg=pos+zero]
PASS [DD no ovf + zero]
PASS [SUB normal]
PASS [ ovf neg-pos=pos]
PASS [ ovf pos-neg=neg]
PASS [SUB zero]
PASS [AND normal]
PASS [AND zero]
PASS [OR normal]
PASS [OR zero]
PASS [XOR zero]
PASS [XOR normal]
PASS [NOR all zero in]
PASS [NOR zero out]
PASS [SRL 0x80>>1]
PASS [SRL 0xFF>>8]
PASS [SRL zero]
PASS [SRA 0x80>>>1]
PASS [SRA -1>>>4]
PASS [SRA zero]
PASS [Default op]
=====
Resultado: 23 PASS / 0 FAIL
>>> TODOS LOS TESTS PASARON <<<
=====
tb_alu.sv:192: $finish called at 23000 (1ps)
=== Simulación finalizada correctamente ===
```

Figure 4: Resultado de la simulación de tb_alu

5. Limpieza del registro mediante reset.



```
vale@ThinkBook-14: ~/repos/arq-d-comp/TP1
vare@ThinkBook-14:~/repos/arq-d-comp/TP1$ ./test.sh btn
=== Testbench btn_reg ===
btn_reg.sv:1: warning: timescale for btn_reg inherited from another file.
tb_btn_reg.sv:1: ...: The inherited timescale is here.
=== Inicio testbench btn_reg ===
PASS [et initial state]
PASS [e on rising edge]
PASS [hile button held]
PASS [nd capture works]
PASS [eset clears data]
=====
Resultado: 5 PASS / 0 FAIL
>>> TODOS LOS TESTS PASARON <<<
=====
tb_btn_reg.sv:79: $finish called at 87000 (1ps)
=== Simulación finalizada correctamente ===
```

Figure 5: Resultado de la simulación de `tb_btn_reg`

9.3. Testbench de integración

`tb_top.sv` verifica la interacción entre los registros, la ALU y los LEDs.

Los casos implementados son:

Prueba	Funcionamiento verificado
Test 1	$3 + 5 = 8$
Test 2	$0x7F + 1$, detección de overflow
Test 3	$5 - 5 = 0$, detección de Zero
Test 4	Reset de los valores capturados
Test 5	Captura solamente en el flanco ascendente
Test 6	No recaptura mientras el botón permanece presionado

10. Decisiones de diseño

10.1. Parametrización del ancho de datos

La ALU y el registro `btn_reg` utilizan parámetros para definir su ancho.

El `top` utiliza:

`NB_DATA = 8`

por lo que los operandos y el resultado son de 8 bits.

Esta parametrización permite reutilizar los módulos para otros anchos de datos sin modificar la estructura general.

```
vale@ThinkBook-14: ~/repos/arq-d-comp/TP1
vale@ThinkBook-14:~/repos/arq-d-comp/TP1$ ./test.sh top
=== Testbench top ===
top.sv:1: warning: timescale for top inherited from another file.
tb_top.sv:1: ...: The inherited timescale is here.
btn_reg.sv:1: warning: timescale for btn_reg inherited from another file.
tb_top.sv:1: ...: The inherited timescale is here.
alu.sv:1: warning: timescale for alu inherited from another file.
tb_top.sv:1: ...: The inherited timescale is here.
=== Inicio testbench TOP ===
--- Test 1: ADD 3+5=8 ---
PASS [LED ADD 3+5]
--- Test 2: ADD overflow 0x7F+1 ---
PASS [ overflow result]
--- Test 3: SUB 5-5 => zero ---
PASS [LED zero flag]
--- Test 4: reset síncrono ---
PASS [ captured values]
--- Test 5: edge detection ---
PASS [y on rising edge]
--- Test 6: no recapture while held ---
PASS [pture while held]
=====
Resultado: 6 PASS / 0 FAIL
>>> TODOS LOS TESTS PASARON <<<
=====
tb_top.sv:137: $finish called at 736000 (1ps)

=== Simulación finalizada correctamente ===
vale@ThinkBook-14:~/repos/arq-d-comp/TP1$ |
```

Figure 6: Resultado de la simulación de `tb_top`

10.2. Captura por flanco de botón

En lugar de utilizar directamente el pulsador como reloj, el diseño utiliza el reloj de 100 MHz de la FPGA y detecta el flanco ascendente del botón.

Esta decisión permite mantener un único reloj de sistema y hace que la captura ocurra únicamente cuando el botón cambia de estado de liberado a presionado.

El registro `r_btn_prev` almacena el estado anterior del botón para realizar esta detección.

10.3. Reset

El reset se implementa de forma síncrona. Cuando `btnR` está activo durante un flanco del reloj, los registros de operandos y operación se limpian.

La ALU es combinacional, por lo que al quedar los registros en cero, el resultado correspondiente también se actualiza.

10.4. Tratamiento del overflow

Se eligió detectar overflow signed en complemento a dos para `ADD` y `SUB`.

No se implementó saturación. Por lo tanto, el resultado mantiene los bits correspondientes a la operación de 8 bits y la flag `Overflow` informa que el valor matemático excedió el rango representable.

10.5. Ausencia de Carry

No se agregó una flag `Carry`, dado que las flags implementadas por este diseño son `Zero` y `Overflow`.

10.6. Código de operación

Los ocho códigos de operación se definen mediante `localparam`, evitando utilizar directamente valores numéricos dentro de cada decisión del `case`.

Esto mejora la legibilidad y permite relacionar cada operación con su código de manera explícita.

11. Conclusiones

La implementación permitió desarrollar una ALU parametrizable utilizando SystemVerilog y trasladar su funcionamiento a una FPGA Digilent Basys3.

La separación del diseño en módulos permite distinguir claramente la lógica combinacional de la lógica secuencial. La ALU se encarga exclusivamente de realizar las operaciones y generar las flags, mientras que los registros almacenan los operandos y el código de operación antes de enviarlos a la unidad de procesamiento.

La detección del flanco ascendente de los pulsadores permite utilizar los botones como mecanismo de captura sin convertirlos directamente en señales de reloj. De esta manera, los datos ingresados mediante los switches permanecen almacenados hasta una nueva captura o hasta realizar un reset.

La implementación de `Zero` y `Overflow` permite además incorporar información de estado a las operaciones, particularmente en los casos aritméticos donde el resultado puede exceder el rango representable con signo.

Finalmente, la inclusión de testbenches independientes y de un script de automatización permite estructurar la verificación en diferentes niveles, facilitando la detección de errores tanto en los módulos individuales como en la integración completa del sistema.